

OPERATING SYSTEMS [BCE26PC01]

UNIT-1

PRAVIN S. GAME, PH.D.

ASSOCIATE PROFESSOR

DEPARTMENT OF COMPUTER ENGINEERING

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

• Teaching Scheme

- Lecture : 02 hours/week
- Laboratory : 02 hours/week
- Credits : 03

• Evaluation Scheme

- FA1 – 10 marks (Problem solving - Unit 1 and 2)
- FA2 – 10 marks (Problem solving - Unit 3 and 4)
- SA – 30 marks
- TW - 50 marks
- Passing criteria : SA - 40% AND TW - 40% AND (FA+SA+TW) - 40%

• Prerequisite

- Computer Organization, Data Structures and Algorithms

•Course Objectives:

1. To introduce the Operating system and process, threads and CPU scheduling
2. To explore inter process communication mechanisms, to introduce the critical-section problem and solutions.
3. To explore various techniques of allocating memory to processes and explain the concepts of demand paging, page-replacement algorithms.
4. To describe the details of implementing local file systems and directory structures

PRAVIN S. GAME

•Course Outcomes:

After learning the course, the students should be able to:

1. Compare various process scheduling algorithms for a given snapshot of the system.
2. Analyze IPC mechanisms and solutions of process synchronization for critical section problems.
3. Analyze the performance of memory management algorithms for a given problem.
4. Apply disk scheduling policies for a given I/O request sequence with file management concepts

PRAVIN S. GAME

UNIT -1

•Introduction to Operating Systems

Introduction to Operating Systems: Introduction, Evolution, Types, Services, system calls.

• Process Management:

Process Concept -Process states, Process control block, Threads -Introduction, POSIX / pthreads. Process Scheduling: Basic Concepts, Scheduling Criteria, Scheduling Algorithms.

PRAVIN S. GAME

COMPUTER : BASIC ELEMENTS

Processor

**I/O
Modules**

**Main
Memory**

**System
Bus**



PRAVIN S. GAME

INTRODUCTION

Operating + System

Operate : function;
Control the functioning of
a machine or activities of
an organization.

A set of things that
are connected or
that work together.

PRAVIN S. GAME

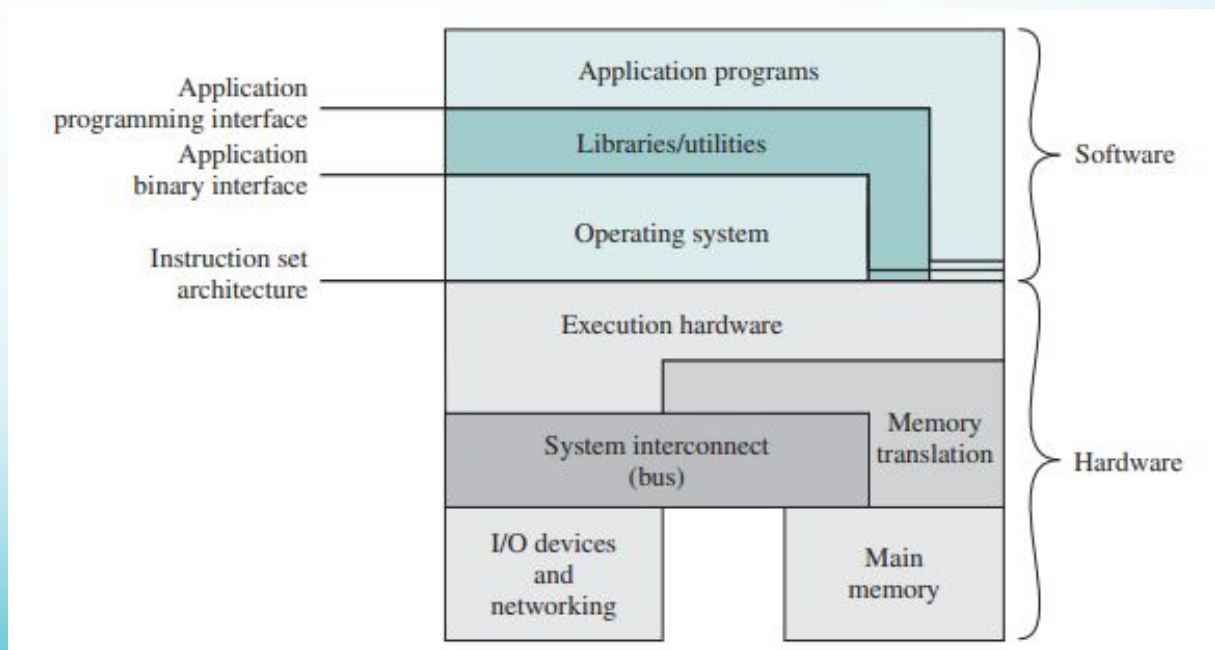
OPERATING SYSTEM

- A program that controls the execution of application programs
- An interface between applications and hardware

Main objectives of an OS:

- Convenience
- Efficiency
- Ability to evolve

PRAVIN S. GAME



Computer Hardware and Software Structure

PRAVIN S. GAME

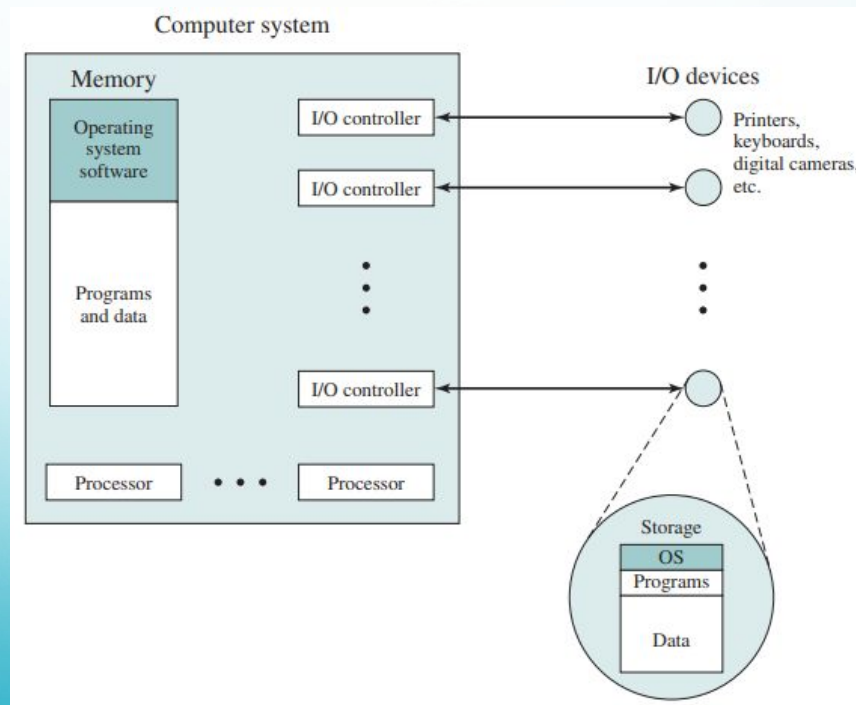
FUNCTIONS OF OS

- The Operating System as a User/Computer Interface

- Program development
- Program execution
- Access I/O devices
- Controlled access to files
- System access
- Error detection and response
- Accounting

PRAVIN S. GAME

• The Operating System as Resource Manager



Resources:

1. I/O devices
2. Processor
3. Primary Memory
4. Secondary Memory

PRAVIN S. GAME

EVOLUTION OF OPERATING SYSTEMS

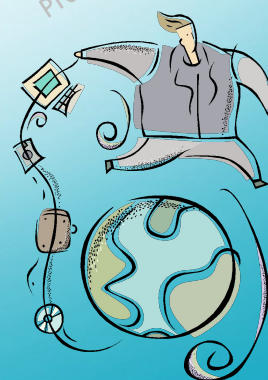
- A major OS will evolve over time for a number of reasons:

hardware upgrades

new types of hardware

new services

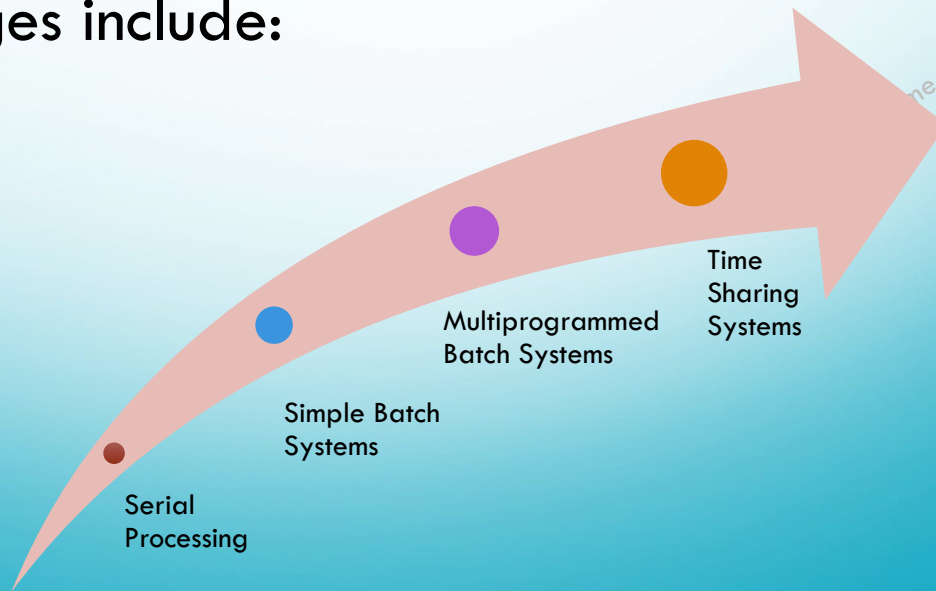
Fixes



PRAVIN S. GAME

EVOLUTION OF OPERATING SYSTEMS

■ Stages include:



PRAVIN S. GAME

SERIAL PROCESSING

□ Earliest Computers:

- No operating system
 - programmers interacted directly with the computer hardware
- Computers ran from a console with display lights, toggle switches, some form of input device, and a printer
- Users have access to the computer in “series”

□ Problems:

- Scheduling:
 - most installations used a hardcopy sign-up sheet to reserve computer time
 - time allocations could run short or long, resulting in wasted computer time
- Setup time
 - a considerable amount of time was spent just on setting up the program to run

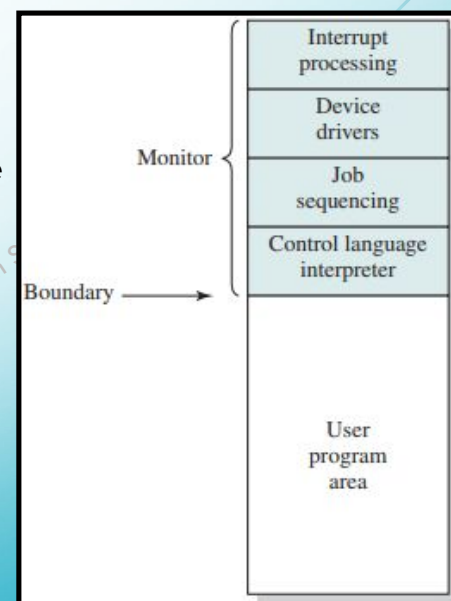
PRAVIN S. GAME

SIMPLE BATCH SYSTEMS

- Early computers were very expensive
 - ✓ important to maximize processor utilization
- Monitor (Batch OS)
 - ✓ user no longer has direct access to processor
 - ✓ job is submitted to computer operator who batches them together and places them on an input device
 - ✓ program branches back to the monitor when finished
 - ✓ Job Control Language
- ✓ Two point of views : Monitor and Processor

PRAVIN S. GAME

- Monitor point of view:
 - Controls the sequence of events
 - The monitor reads in jobs one at a time from the input device
 - As it is read in, the current job is placed in the user program area, and control is passed to this job.
 - When the job is completed, it returns control to the monitor.
 - The results of each job are sent to an output device



PRAVIN S. GAME

- Processor point of view

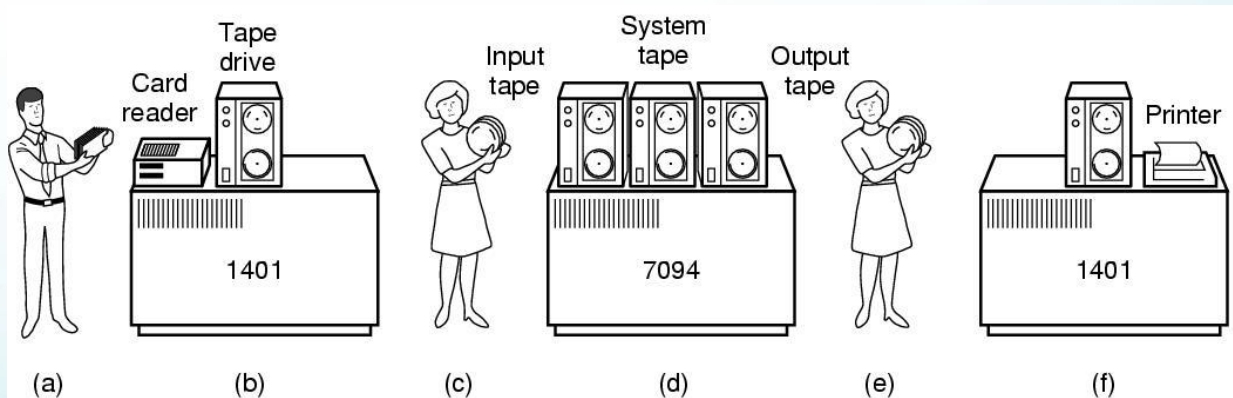
- The processor executes instructions from main memory containing the monitor.
- “control is passed to a job”
- “control is returned to the monitor”

- The batch OS, is simply a computer program

- Desirable H/W features:

- Memory Protection
- Timer
- Privileged Instructions
- Interrupts

PRAVIN S. GAME



- (a) Programmers bring cards to 1401
- (b) 1401 reads batch of jobs onto tape
- (c) Operator carries input tape to 7094
- (d) 7094 does computing
- (e) Operator carries output tape to 1401
- (f) 1401 prints output.

PRAVIN S. GAME

MODES OF OPERATION

User Mode

- user program executes in user mode
- certain areas of memory are protected from user access
- certain instructions may not be executed

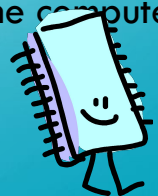
Kernel Mode

- monitor executes in kernel mode
- privileged instructions may be executed
- protected areas of memory may be accessed

PRAVIN S. GAME

SIMPLE BATCH SYSTEM OVERHEAD

- Processor time alternates between execution of user programs and execution of the monitor
- Sacrifices:
 - some main memory is now given over to the monitor
 - some processor time is consumed by the monitor
- Despite overhead, the simple batch system improves utilization of the computer



PRAVIN S. GAME

MULTIPROGRAMMED BATCH SYSTEMS

- Processor is often idle
- even with automatic job sequencing
- I/O devices are slow compared to processor

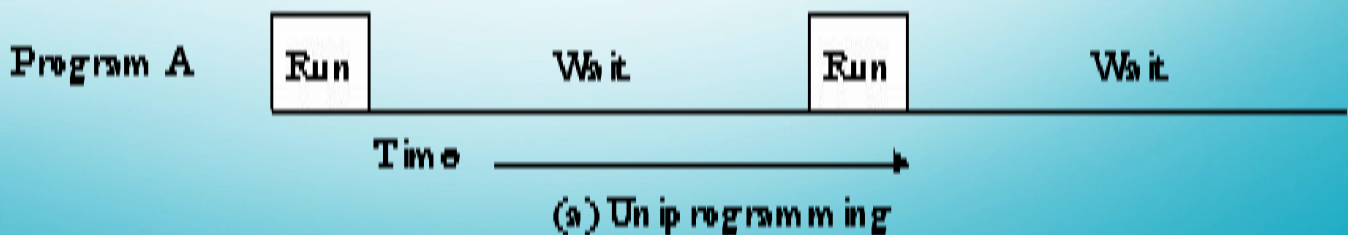
Read one record from file	15 μ s
Execute 100 instructions	1 μ s
Write one record to file	15 μ s
TOTAL	31 μ s

$$\text{Percent CPU Utilization} = \frac{1}{31} = 0.032 = 3.2\%$$

PRAVIN S. GAME

• UNIPROGRAMMING

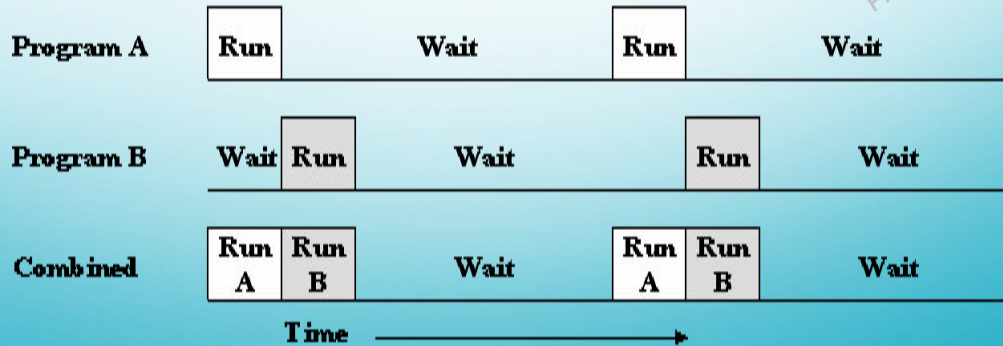
- The processor spends a certain amount of time executing, until it reaches an I/O instruction; it must then wait until that I/O instruction concludes before proceeding



PRAVIN S. GAME

MULTIPROGRAMMING

- There must be enough memory to hold the OS (resident monitor) and one user program
- When one job needs to wait for I/O, the processor can switch to the other job, which is likely not waiting for I/O

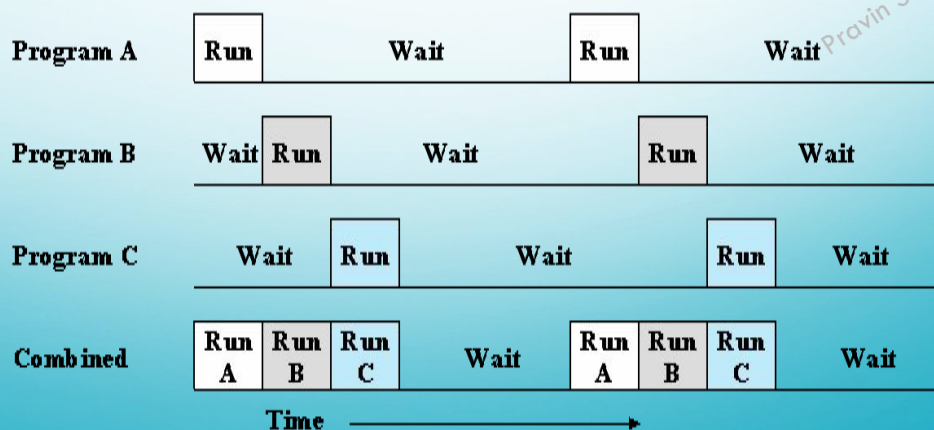


(b) Multiprogramming with two programs

PRAVIN S. GAME

MULTIPROGRAMMING

- Aka multitasking
- Memory is expanded to hold three, four, or more programs and switch among all of them



(c) Multiprogramming with three programs

PRAVIN S. GAME

TIME-SHARING SYSTEMS

- Can be used to handle multiple interactive jobs
- Processor time is shared among multiple users
- Multiple users simultaneously access the system through terminals, with the OS interleaving the execution of each user program in a short burst or quantum of computation

PRAVIN S. GAME

BATCH MULTIPROGRAMMING VS. TIME SHARING

	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

PRAVIN S. GAME

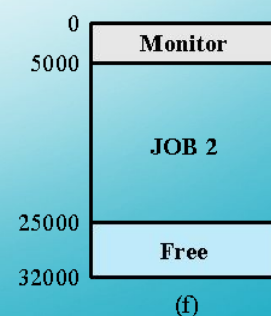
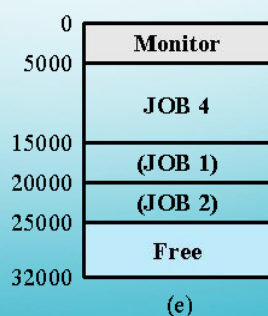
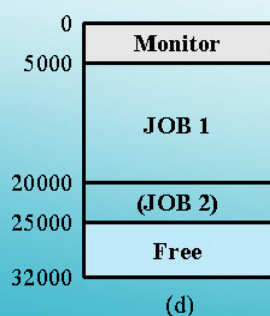
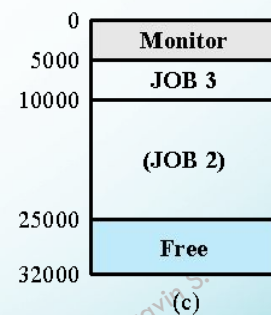
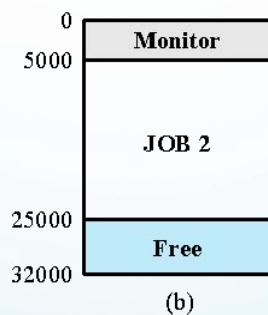
COMPATIBLE TIME-SHARING SYSTEMS

- One of the first time-sharing operating systems developed at MIT
- Ran on a computer with 32,000 36-bit words of main memory, with the resident monitor consuming 5000 of that
- To simplify both the monitor and memory management a program was always loaded to start at the location of the 5000th word

✓ Time Slicing

- System clock generates interrupts at a rate of approximately one every 0.2 seconds
- At each interrupt OS regained control and could assign processor to another user
- Preemption was used
- Old user programs and data were written out to disk
- Old user program code and data were restored in main memory when that program was next given a turn

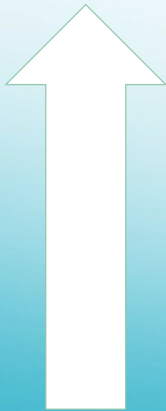
PRAVIN S. GAME



PRAVIN S. GAME

MAJOR ACHIEVEMENTS

- Operating Systems are among the most complex pieces of software ever developed



Major advances in development include:

- processes
- memory management
- information protection and security
- scheduling and resource management
- system structure

PRAVIN S. GAME

• Machine Language

1300042774

1400593419

1200274027

• High-Level Language

grossPay = basePay + overTimePay

• Assembly language

LOAD BASEPAY

ADD OVERPAY

STORE GROSSPAY

PRAVIN S. GAME

OPERATING SYSTEM DEFINITION

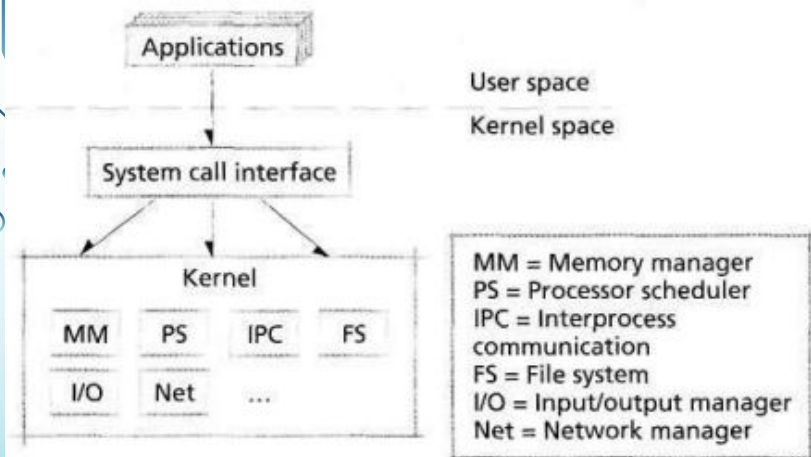
- OS is a **resource allocator**
 - Manages all resources
 - Decides between conflicting requests for efficient and fair resource use
- OS is a **control program**
 - Controls execution of programs to prevent errors and improper use of the computer
- “The one program running at all times on the computer” is the **kernel**. Everything else is either a system program (ships with the operating system) or an application program.

PRAVIN S. GAME

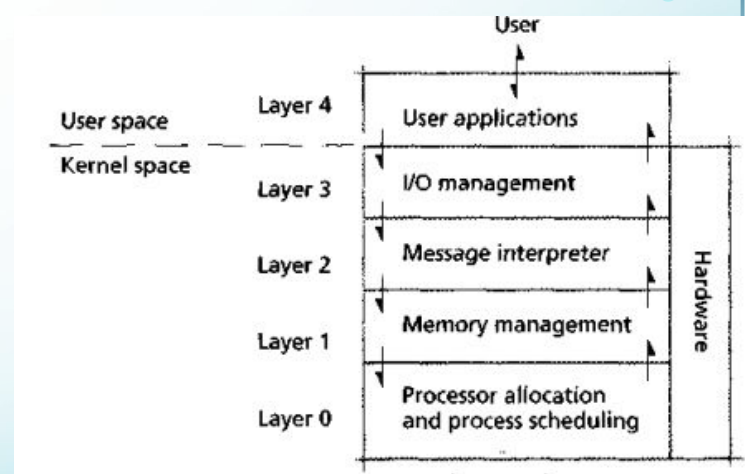
OS BASED ON ARCHITECTURE

- Monolithic : OS/360, VMS, Linux, Android
- Layered : THE OS, Linux, Windows XP ...
- Microkernel : Minix, Amoeba, JavaOS
- Networked and Distributed OS : Cisco NX-OS, NVIDIA Cumulus Linux, SONic, MS Windows Server..... Solaris, Mach/OS, Amoeba

PRAVIN S. GAME

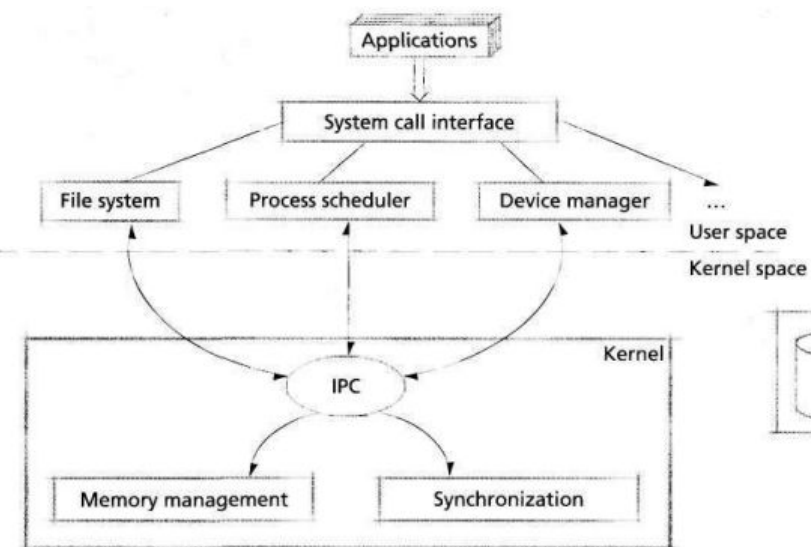


Monolithic

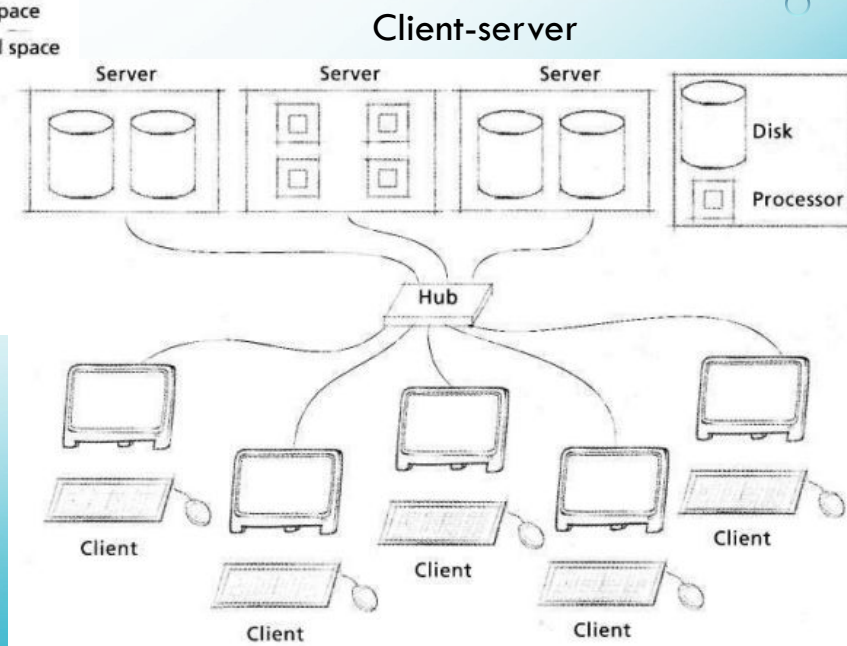


Layered

PRAVIN S. GAME



Microkernel



Client-server

PRAVIN S. GAME

OPERATING SYSTEM COMPONENTS

Core Components

- Process management
- Memory management,
- Cache mgmt
- I/O Management
- IPC
- File Management
- Process scheduler and dispatcher
- Secondary/Mass storage mgmt
- Device drivers
- Disk scheduler

PRAVIN S. GAME

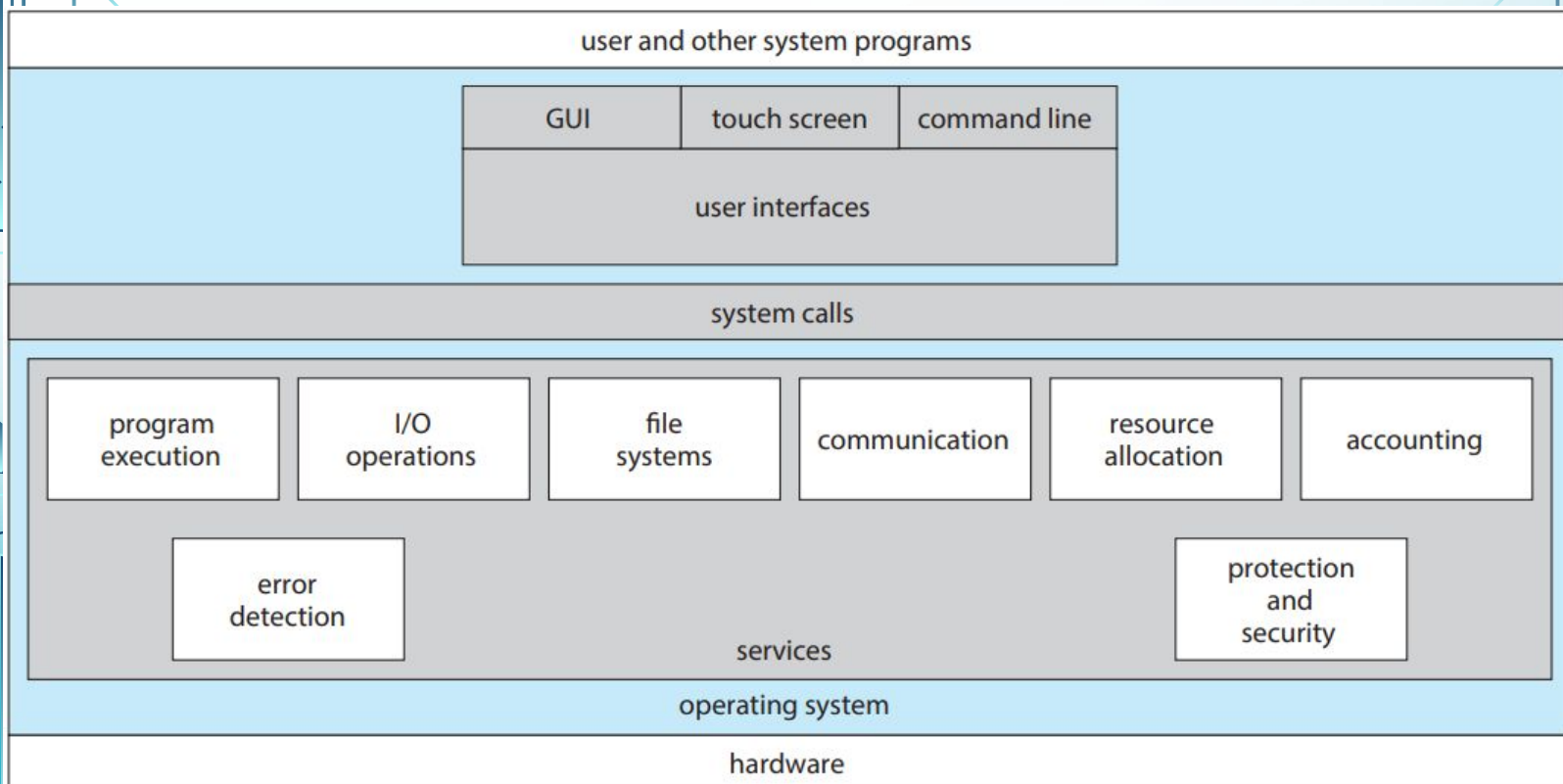
Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

PRAVIN S. GAME

OPERATING SYSTEM SERVICES

- User Interface
- Program Execution
- I/O Operations
- File System Manipulation
- Communications
- Error Detection
- Resource allocation
- Logging
- Protection and security

PRAVIN S. GAME



PRAVIN S. GAME

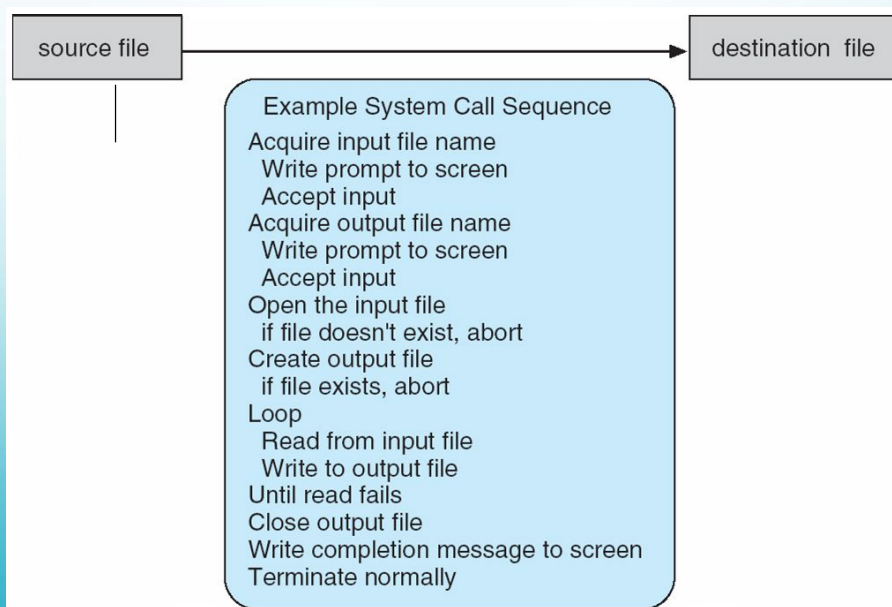
SYSTEM CALLS

- Programming interface to the services provided by the OS
- Typically written in C or C++ or assembly language
- Mostly accessed by programs via a high-level **Application Program Interface (API)** rather than direct system call use
- Three most common APIs are
 - Win32 API for Windows,
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and
 - Java API for the Java virtual machine (JVM)

PRAVIN S. GAME

EXAMPLE OF SYSTEM CALLS

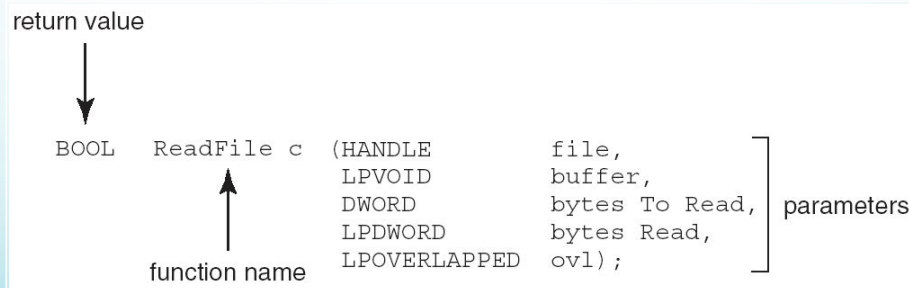
- System call sequence to copy the contents of one file to another file



PRAVIN S. GAME

EXAMPLE OF STANDARD API

- Consider the ReadFile() function in the
- Win32 API—a function for reading from a file



- A description of the parameters passed to ReadFile()
 - HANDLE file—the file to be read
 - LPVOID buffer—a buffer where the data will be read into and written from
 - DWORD bytesToRead—the number of bytes to be read into the buffer
 - LPDWORD bytesRead—the number of bytes read during the last read
 - LPOVERLAPPED ovl—indicates if overlapped I/O is being used

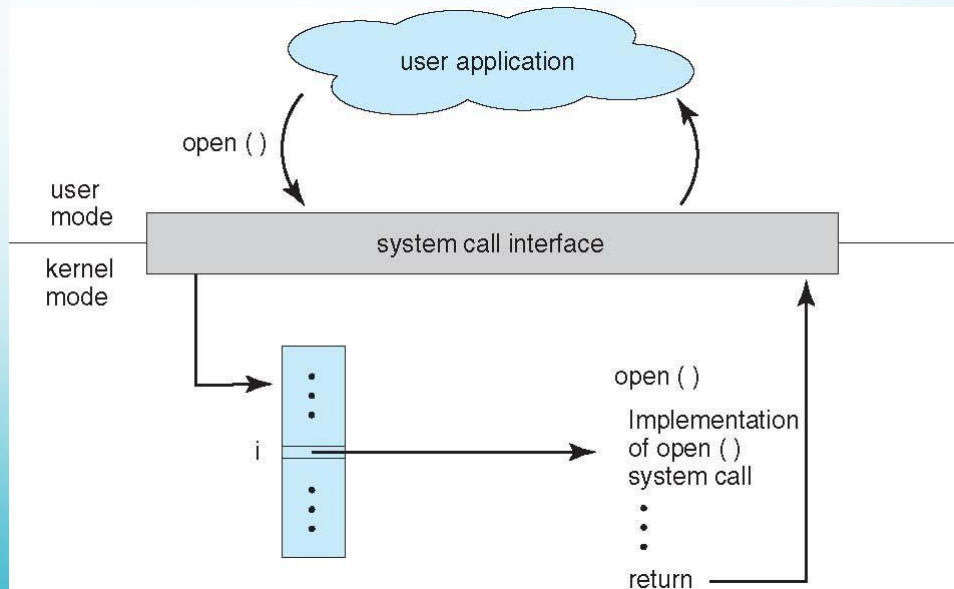
PRAVIN S. GAME

SYSTEM CALL IMPLEMENTATION

- Typically, a number is associated with each system call
 - System-call interface maintains a table indexed according to these numbers
- The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result to call
 - Most details of OS interface hidden from programmer by API
 - Managed by run-time support library (set of functions built into libraries included with compiler)

PRAVIN S. GAME

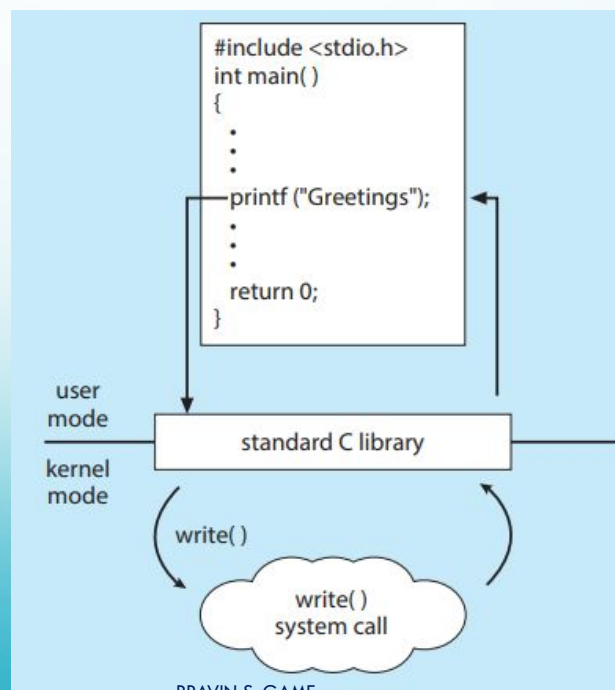
API – SYSTEM CALL – OS RELATIONSHIP



PRAVIN S. GAME

STANDARD C LIBRARY EXAMPLE

- C program invoking printf() library call, which calls write() system call



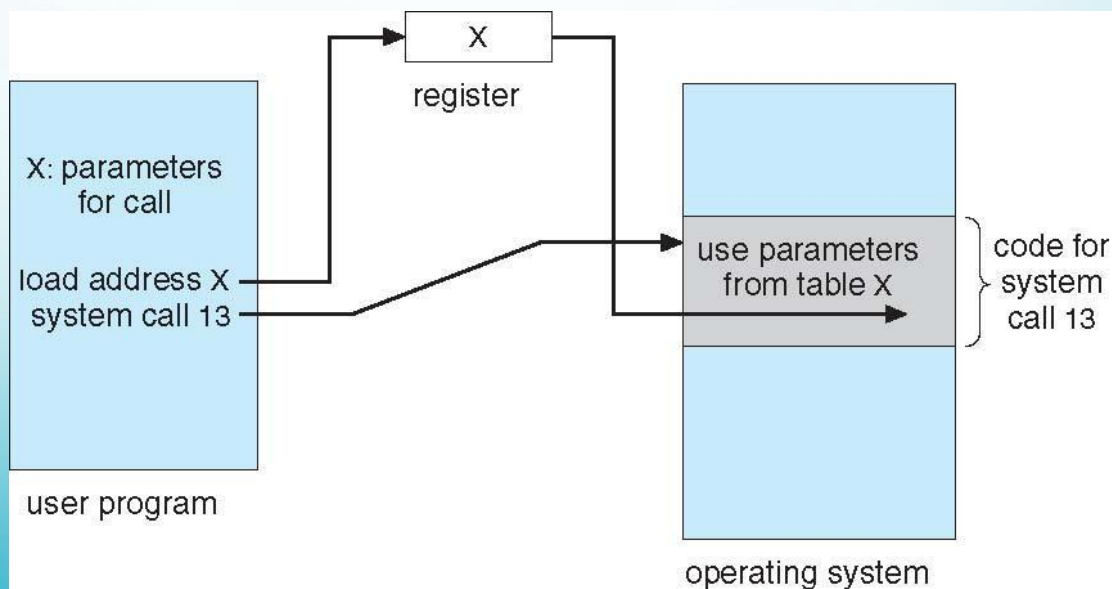
PRAVIN S. GAME

SYSTEM CALL PARAMETER PASSING

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - Simplest: pass the parameters in *registers*
 - In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - Linux and Solaris
 - Parameters placed, or *pushed*, onto the stack by the program and *popped off* the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed

PRAVIN S. GAME

PARAMETER PASSING VIA TABLE



PRAVIN S. GAME

TYPES OF SYSTEM CALLS

- Process control

- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

- File management

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes

PRAVIN S. GAME

TYPES OF SYSTEM CALLS ...

- Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

- Information maintenance

- get time or date, set time or date
- get system data, set system data
- get and set process, file, or device attributes

- Communications

- create, delete communication connection
- send, receive messages
- transfer status information
- attach and detach remote devices

- Protection

- Set/get permission
- Allow/deny user

PRAVIN S. GAME

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

PRAVIN S. GAME

THINK-PAIR-SHARE

- Why is it dangerous to allow users to perform read or write operations to any region of disk at will?
- How does device driver support contribute to an operating system's extensibility?
- Why must processes issue system calls to request operating system services?
- Why use APIs rather than system calls?
- Why are system calls necessary for an operating system?

PRAVIN S. GAME

QUIZ

Which operating system components perform each of the following operations?

- a. Write to disk.
- b. Determine which process will run next.
- c. Determine where in memory a new process should be placed.
- d. Organize files on disk.
- e. Enable one process to send data to another.

PRAVIN S. GAME

PROCESS - CONCEPTS

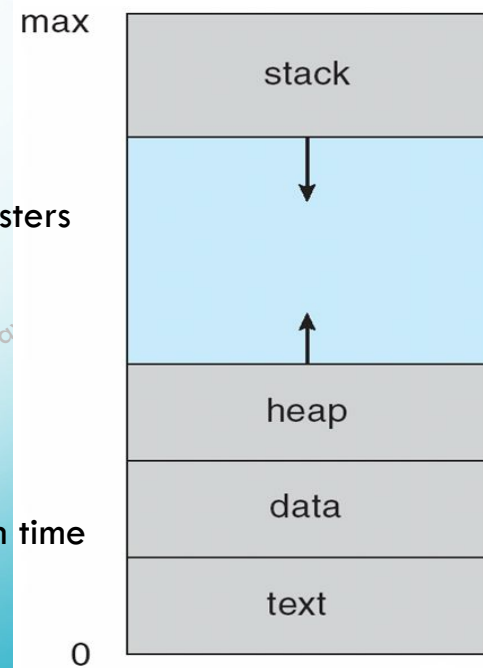
- A Program in execution
- An active entity
- Term was first used by the designers of Multics (1960s)
- the "animated spirit" of a procedure
- that entity to which processors are assigned
- Dispatchable unit

Program : Process :: Sheet music : Symphony orchestra

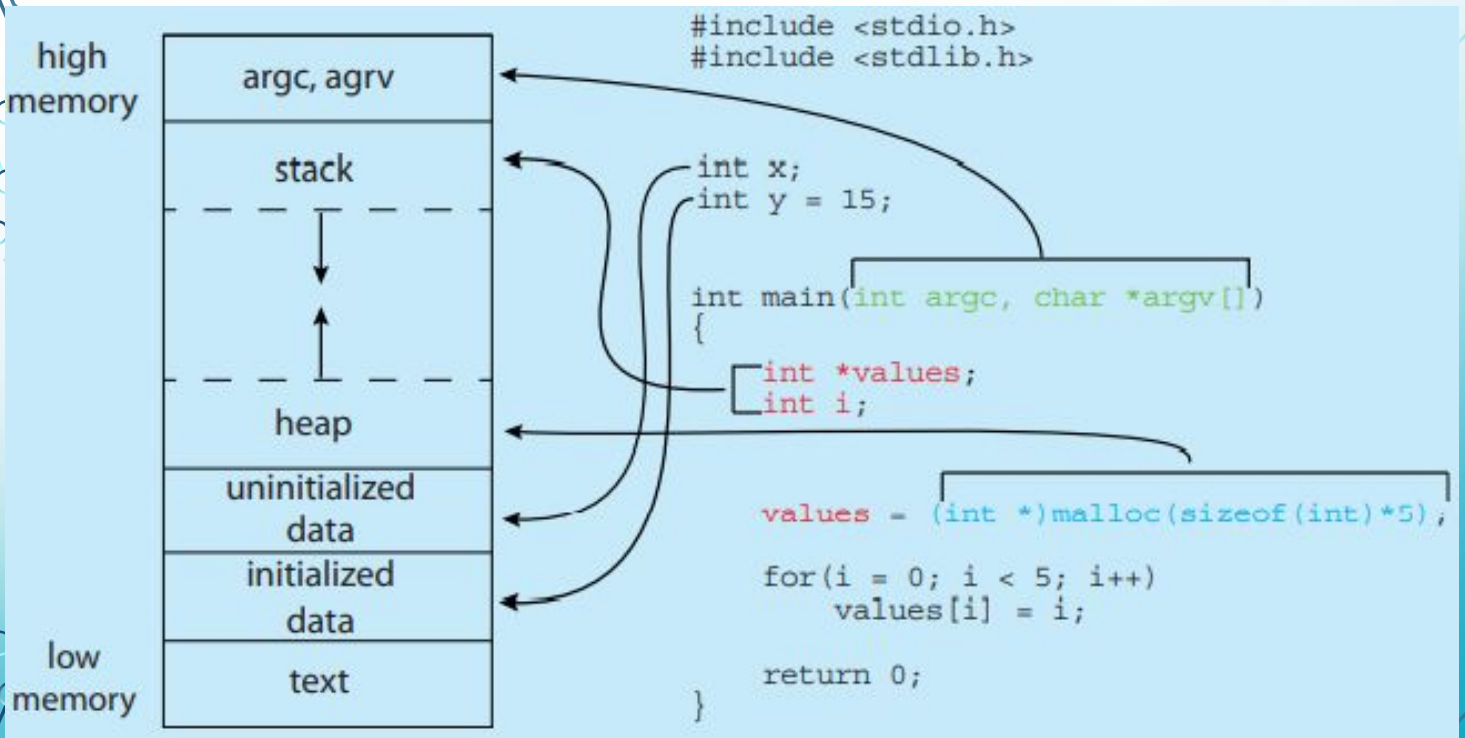
PRAVIN S. GAME

CONCEPTS

- The program code, also called **text section**
- Current activity including **program counter**, processor registers
- **Stack** containing temporary data
 - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time



PRAVIN S. GAME



C program in memory

PRAVIN S. GAME

CONCEPTS

Creation

- During System initialization
- By system call by a running process
- By user
- Initialization of a batch job

Process Hierarchies

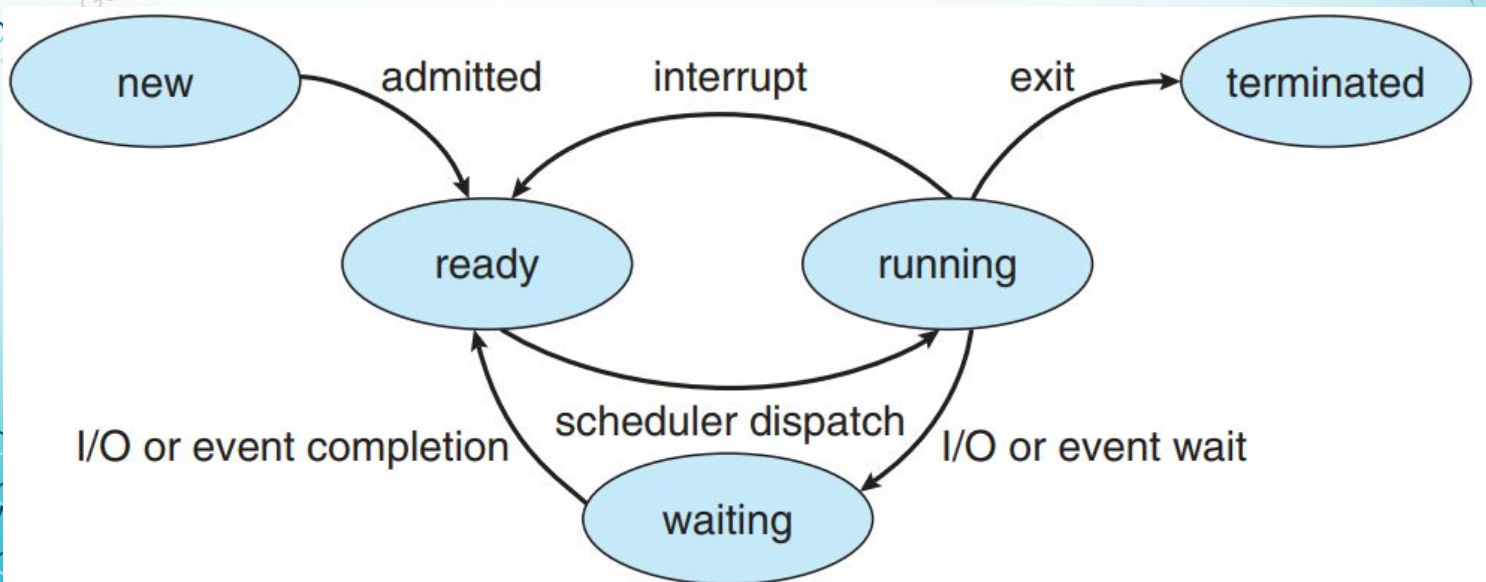
Process States...

Termination

- Voluntary exit
 - Normal
 - Error
- Involuntary
 - Fatal error
 - Killed by other process

PRAVIN S. GAME

PROCESS STATES



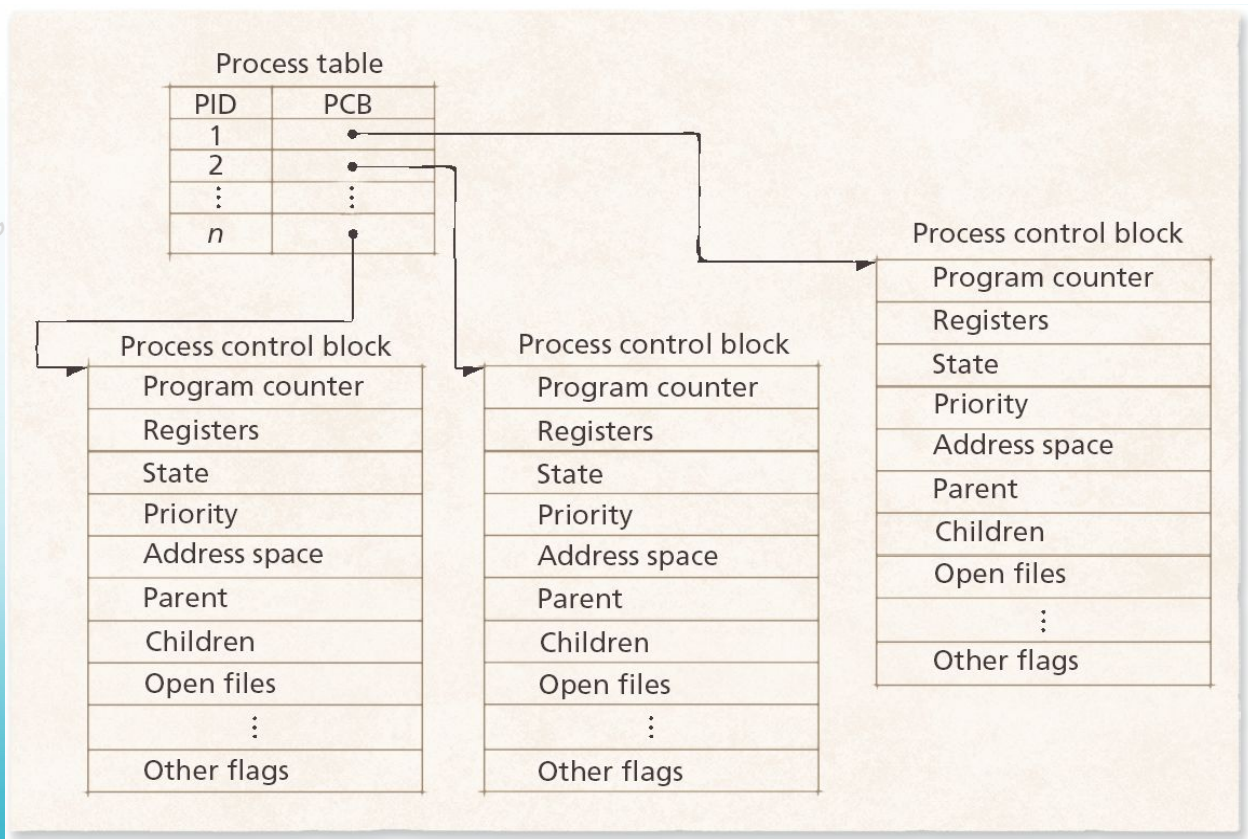
PRAVIN S. GAME

PROCESS CONTROL BLOCK

- Contains information about the process

Process management	Memory management	File management
Process Id	Pointer to text segment info	Root directory
Program counter	Pointer to data segment info	Working directory
Process state	Pointer to stack segment info	File descriptors
Registers		User ID
Stack pointer		Group ID
Priority		Open files
Scheduling parameters		
Parent process		
Process group		
Process start and end time		
CPU time used		

PRAVIN S. GAME



PRAVIN S. GAME

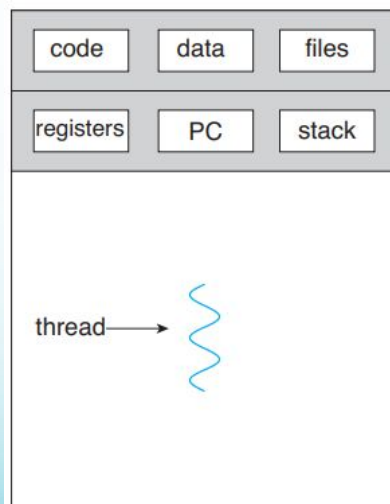
PROCESS OPERATIONS

- Create
- Destroy
- Suspend
- Resume
- Change priority
- Block
- Wake up
- Dispatch
- Communicate with other process

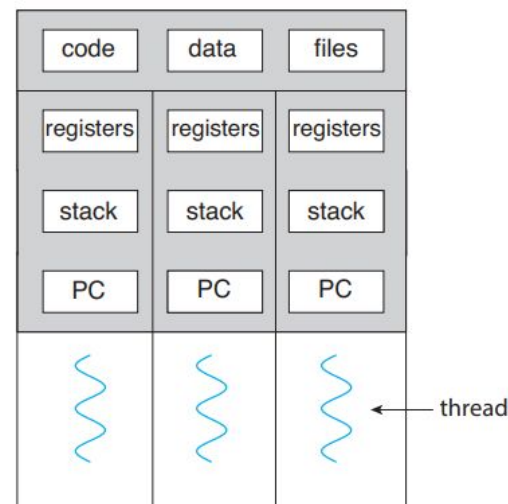
PRAVIN S. GAME

THREADS

- a basic unit of CPU utilization
- It has a thread ID, a program counter (PC), a register set, and a stack



single-threaded process



multithreaded process

PRAVIN S. GAME

- Most of modern OS and mobile devices are multithreaded.

- **Benefits:**

- Responsiveness
- Resource sharing
- Economy
- Scalability

- **Multithreading models**

- Many-to-one
- One-to-one
- Many-to-many

PRAVIN S. GAME

- **Thread libraries**

- Provides programmer with an API for creating and managing threads
- User-level and kernel-level library

- **Three main libraries**

- POSIX Pthread (user/kernel level)
- Windows threads (kernel level)
- Java threads (depending on host system)

- Windows : Java threads are implemented using Windows API
- UNIX, Linux, and macOS systems : Pthreads

PRAVIN S. GAME

PTHREADS

- Refers to the POSIX standard (IEEE 1003.1c)

PRAVIN S. GAME

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this
function */
int main(int argc, char *argv[])
{
    Pthread_t tid; /* the thread identifier */
    Pthread_attr_t attr; /* set of thread attributes */
    /* set the default attributes of the thread */
    Pthread_attr_init(&attr);
    /* create the thread */
    Pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    Pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}
```

PRAVIN S. GAME


```
/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;
    for (i = 1; i <= upper; i++)
        sum += i;
    Pthread_exit(0);
}
```

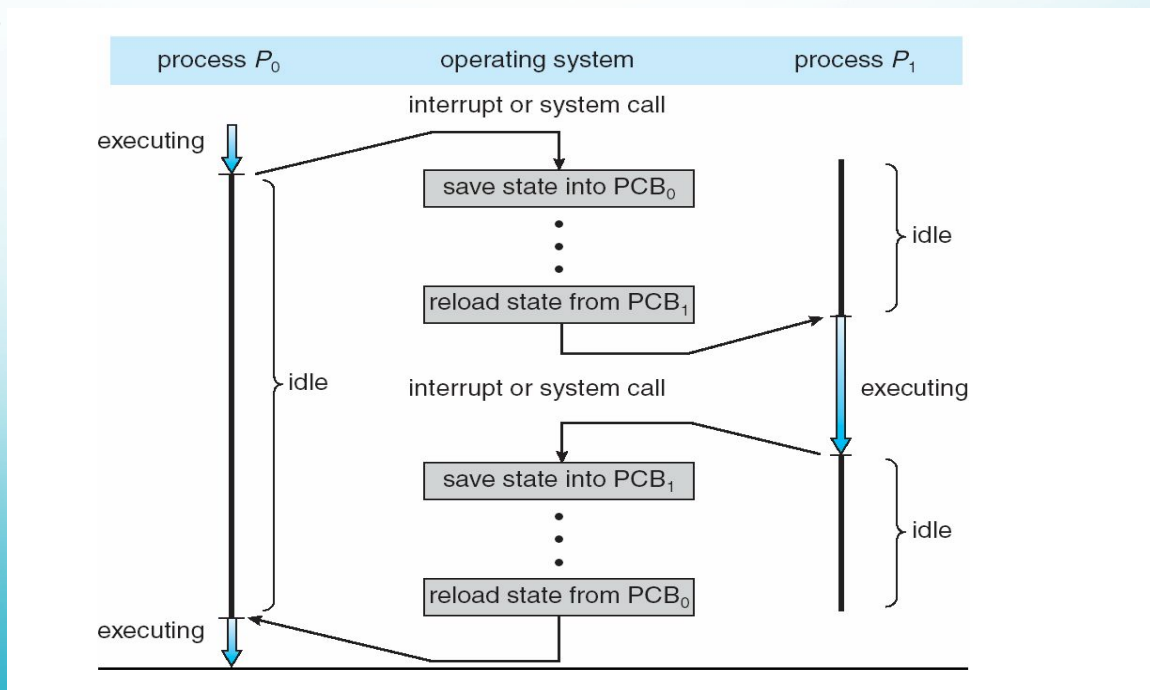
PRAVIN S. GAME

BASIC CONCEPTS

- Maximum CPU utilization obtained with multiprogramming
- CPU-I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- **CPU burst** distribution

PRAVIN S. GAME

CPU SWITCH FROM PROCESS TO PROCESS



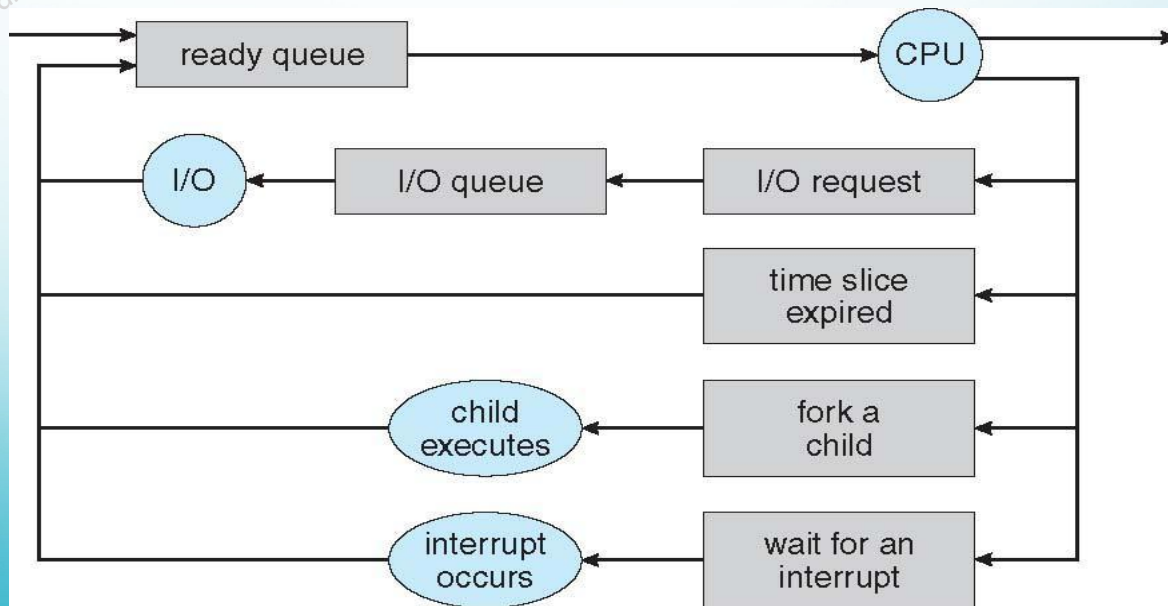
PRAVIN S. GAME

PROCESS SCHEDULING

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
 - **Job queue** – set of all processes in the system
 - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - **Device queues** – set of processes waiting for an I/O device
 - Processes migrate among the various queues

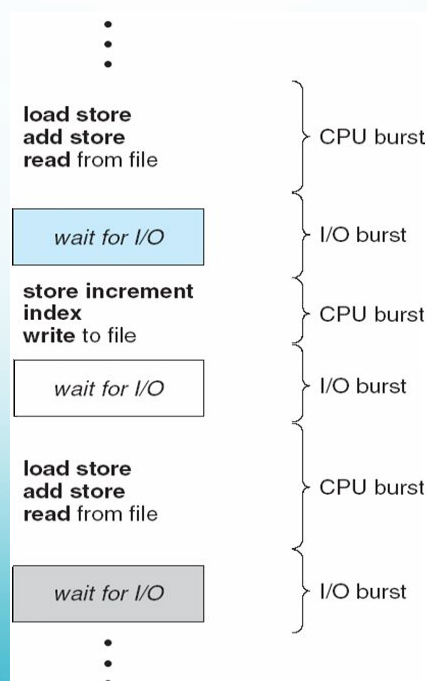
PRAVIN S. GAME

REPRESENTATION OF PROCESS SCHEDULING



PRAVIN S. GAME

ALTERNATING SEQUENCE OF CPU AND I/O BURSTS



PRAVIN S. GAME

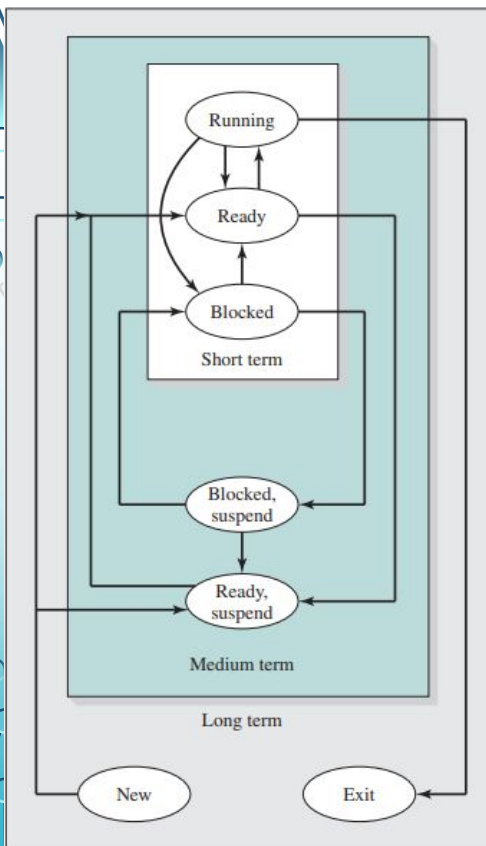
SCHEDULERS

- **Long-term scheduler** (or job scheduler)
- **Short-term scheduler** (or CPU scheduler)
 - Sometimes the only scheduler in a system
- Short-term scheduler is invoked very frequently (milliseconds) $\Rightarrow$ (must be fast)
- Long-term scheduler is invoked very infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
- The long-term scheduler controls the *degree of multiprogramming*
- **Mid-term scheduler**
 - Processes can be described as either:
 - I/O-bound process
 - CPU-bound process

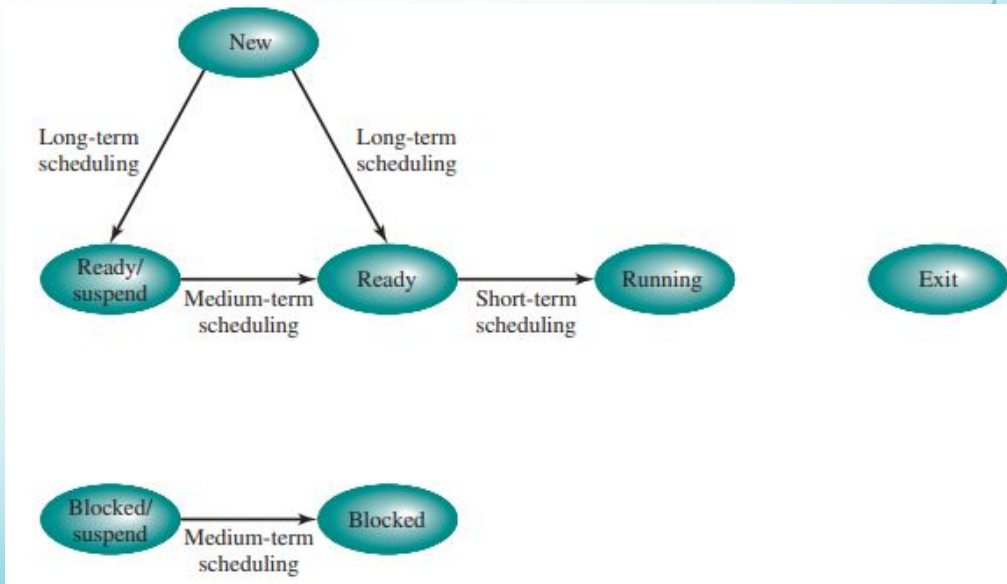
PRAVIN S. GAME

Scheduler	Decision taken
Long-term	To add to the pool of processes to be executed.
Medium-term	To add to the number of processes that are partially or fully in main memory.
Short-term	As to which available process will be executed by the processor.
I/O	As to which process's pending I/O request shall be handled by an available I/O device

PRAVIN S. GAME



Levels of scheduling



Scheduling & Process state transitions

PRAVIN S. GAME

CPU SCHEDULER

- Selects from among the processes in ready queue, and allocates the CPU to one of them
 - Queue may be ordered in various ways
- CPU scheduling decisions may take place when a process:
 1. Switches from running to waiting state
 2. Switches from running to ready state
 3. Switches from waiting to ready
 4. Terminates
- Scheduling under 1 and 4 is **nonpreemptive**
- All other scheduling is **preemptive**
 - Consider access to shared data
 - Consider preemption while in kernel mode
 - Consider interrupts occurring during crucial OS activities

PRAVIN S. GAME

DISPATCHER

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
 - switching context
 - switching to user mode
 - jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

PRAVIN S. GAME

CONTEXT SWITCH

- When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a **context switch**.
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS and the PCB -> longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU -> multiple contexts loaded at once

PRAVIN S. GAME

SCHEDULING CRITERIA

- **CPU utilization**

- **Throughput**

- **Turnaround time**

- **Waiting time**

- **Response time**

For Optimization

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Goals

- Fairness
- Policy enforcement
- Balance
- Ensure predictability
- Scalability

PRAVIN S. GAME

SCHEDULING ALGORITHMS

- **First-Come, First-Served Scheduling**

- **Shortest-Job-First Scheduling**

- **Round-Robin Scheduling**

- **Priority Scheduling**

- **Multilevel Queue Scheduling**

- **Multilevel Feedback Queue Scheduling**

- **Fair-Share Scheduling**

- **Highest-Response Ratio Next scheduling**

PRAVIN S. GAME

```

#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL); //use various exec commands & to be
        //simulated commands
    }
    else { /* parent process */
        /* parent will wait for the child */
        wait (NULL);
        printf ("Child Complete");
    }
    return 0;
}

```

PRAVIN S. GAME

FIRST-COME, FIRST-SERVED SCHEDULING

- Simplest
- FIFO queue
- Coding is simple to write and understand
- Average waiting time is long
 - Ex. (P1, 24), (P2, 3), (P3, 3) ; Avg. waiting time = $(0+24+27)/3 = 17$ milliseconds
- Non-Preemptive
- Convoy effect
 - one CPU-bound process and many I/O-bound processes
 - short process behind long process
- Troublesome for interactive systems

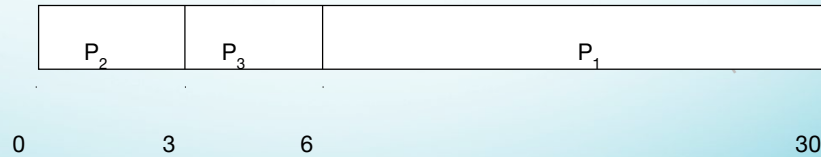
PRAVIN S. GAME

FCFS

Suppose that the processes arrive in the order:

P_2, P_3, P_1

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Turn around time = Completion time – Arrival time
- Waiting time = TAT – Burst time

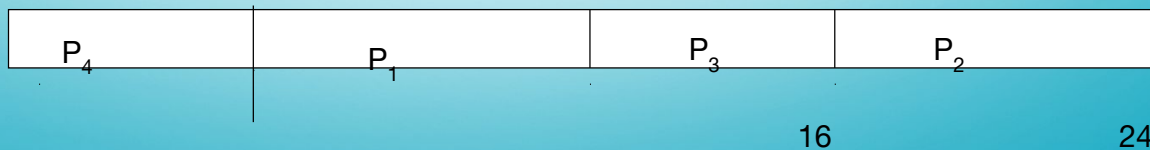
FCFS EXAMPLE

- Five processes arrive at almost same time with estimated running times of 10, 6, 2, 4, and 8 minutes.

Calculate mean process turnaround time and mean waiting time using FCFS scheduling.

SHORTEST-JOB-FIRST SCHEDULING

- SHORTEST-NEXT-CPU-BURST/ Shortest Process Next
- Associate the length of the process's next CPU burst
- Scheduled as per the ascending order of the burst
- Tie ?? Break using FCFS
- Ex. P1, P2, P3, P4, with 6, 8, 7, 3 next burst times



PRAVIN S. GAME

SJF SCHEDULING

- Provably optimal
 - Problem: How to identify the next burst time?
 - Approximate based on the past burst.
 - Predicted as an **exponential average** of the measured lengths of previous CPU bursts.
1. t_n $\equiv$ actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define: $\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$

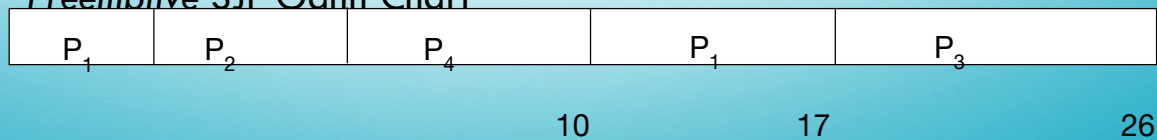
PRAVIN S. GAME

SJF SCHEDULING

- Pre-emptive aka shortest-remaining-time-first
- Non-Preemptive
- Processes with arrival times and preemption to the analysis

	<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
	P_1	0	8
	P_2	1	4
	P_3	2	9
	P_4	3	5

- **Preemptive SJF Gantt Chart**



- Calculate avg. waiting time with and without preemption.

PRAVIN S. GAME

PRIORITY SCHEDULING

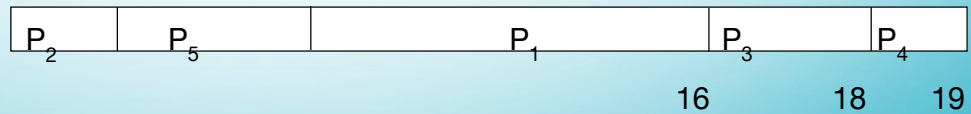
- Priority is associated with each process
- Process with highest priority gets CPU
- Priority can be defined internally or externally.
- Preemptive or non-preemptive
- Problem: Indefinite blocking or starvation
 - Solution : Aging – increase priority of long waiting process

Ex. Given

Process	Burst Time	Priority
---------	------------	----------

P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

• Schedule is



• Avg. waiting time ?

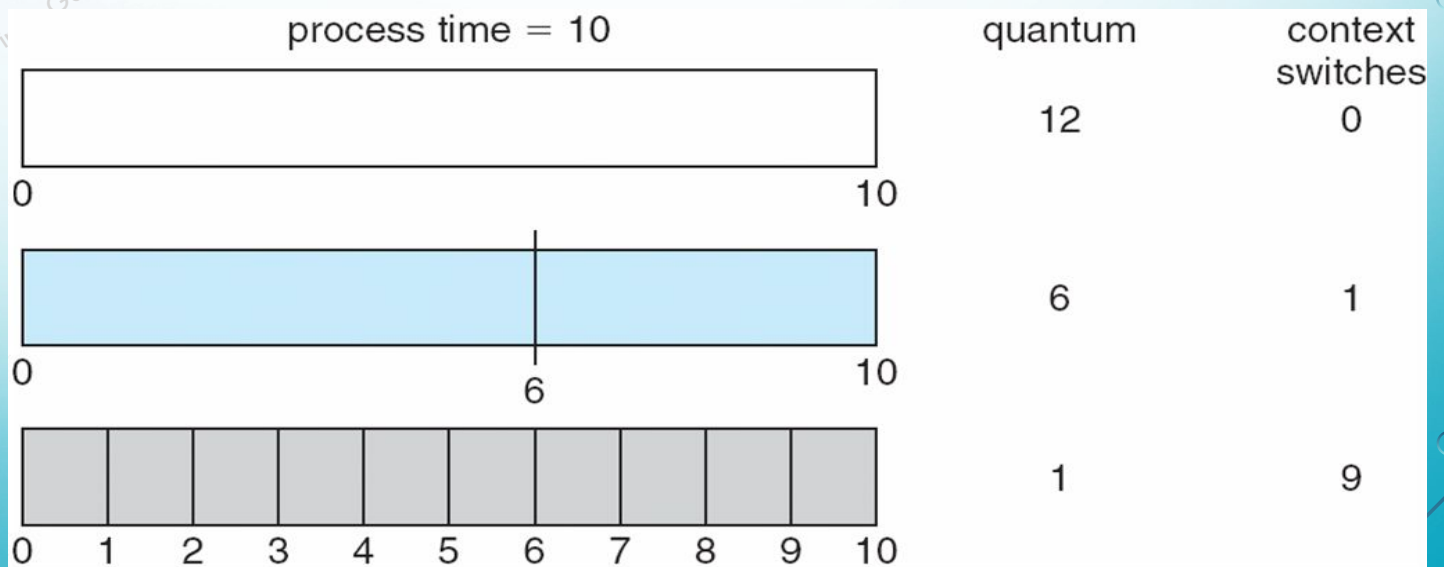
• Avg. Turnaround time ?

PRAVIN S. GAME

ROUND-ROBIN SCHEDULING

- Specially designed for TSS
- FCFS with pre-emption
- A process runs for a defined time slice/quantum.
- Time quantum – 10 to 100 milliseconds
- FIFO ready queue, new or pre-empted process added at tail
- Avg. waiting time – often long
- Performance depends on size of quanta (diag. on next slide)
 - Very Large – FCFS
 - Very small – more context switching
- Context switching time also affects the performance

PRAVIN S. GAME



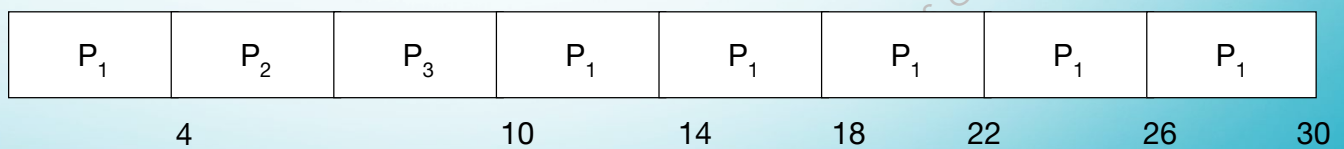
Ex. Given Process Burst Time

P_1 24

P_2 3

P_3 3

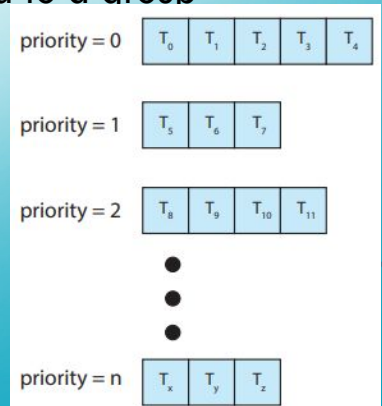
• Schedule



• Avg. waiting time $[(10-4) + 4 + 7] / 3 = 5.66$

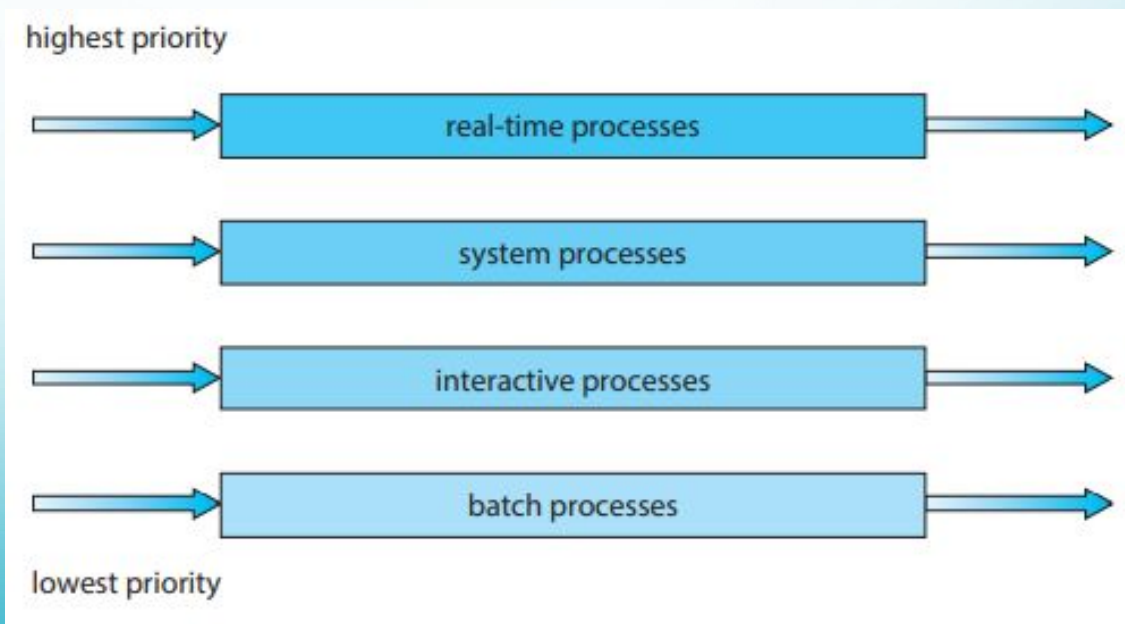
MULTILEVEL QUEUE SCHEDULING

- Where Processes can be partitioned into groups (queue)
- Foreground and background process
- Based on properties of a process, it is permanently assigned to a group
- Each queue has its own scheduling algo
- Among the queues – Fixed priority preemptive scheduling



PRAVIN S. GAME

- Ex.



PRAVIN S. GAME

- Consider the following workload:

Process	Burst Time	Priority	Arrival Time
P1	50 ms	4	0 ms
P2	20 ms	1	20 ms
P3	100 ms	3	40 ms
P4	40 ms	2	60 ms

- 1) Show the schedule using shortest remaining time, nonpreemptive priority (a smaller priority number implies higher priority) and round robin with quantum 30 ms.
- 2) Calculate average waiting time and avg turnaround time for each scheduling policy.

PRAVIN S. GAME

- What factors determine the time quantum in round robin scheduling? Consider the system:

Process	Arrival Time	Processing Time
P1	0	12
P2	2	6
P3	8	18
P4	10	4

Context switch takes a time of 1 unit. Compute the average turnaround time of the processes for the time quanta $q = 2$, $q = 4$, and $q = 8$ respectively.

PRAVIN S. GAME

• Five batch jobs, A through E, arrive at a computer center at essentially the same time. They have an estimated running time of 15, 9, 3, 6, and 12 minutes, respectively. Their (externally defined) priorities are 6, 3, 7, 9, and 4, respectively, with a lower value corresponding to a higher priority. For each of the following scheduling algorithms, determine the turnaround time for each process and the average turnaround for all jobs. Ignore process switching overhead. Explain how you arrived at your answers. In the last three cases, assume only one job at a time runs until it finishes, and all jobs are completely processor bound.

- round robin with a time quantum of 1 minute
- priority scheduling
- FCFS (run in order 15, 9, 3, 6, and 12)
- shortest job first

PRAVIN S. GAME

• Suppose that the following processes arrive for execution at the times indicated. Each process will run for the amount of time listed. In answering the questions, use nonpreemptive scheduling, and base all decisions on the information you have at the time the decision must be made.

Process	AT	BT
P1	0.0	8
P2	0.4	4
P3	1.0	1

- What is the average turnaround time for these processes with the FCFS scheduling algorithm?
- What is the average turnaround time for these processes with the SJF scheduling algorithm?
- The SJF algorithm is supposed to improve performance, but notice that we chose to run process P1 at time 0 because we did not know that two shorter processes would arrive soon. Compute what the average turnaround time will be if the CPU is left idle for the first 1 unit and then SJF scheduling is used.

PRAVIN S. GAME

- Consider the following set of processes, with the length of the CPU burst time given in milliseconds:

Process	BT	Priority
P1	2	2
P2	1	1
P3	8	4
P4	4	2
P5	5	3

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, nonpreemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting time (over all processes)?

PRAVIN S. GAME

- The following processes are being scheduled using a preemptive, round-robin scheduling algorithm.

Process	Priority	Burst	Arrival
P1	40	20	0
P2	30	25	25
P3	30	25	30
P4	35	15	60
P5	5	10	100
P6	10	10	105

Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. In addition to the processes listed below, the system also has an idle task (which consumes no CPU resources and is identified as P_{idle}). This task has priority 0 and is scheduled whenever the system has no other available processes to run time quantum is 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- Show the scheduling order of the processes using a Gantt chart.
- What is the turnaround time for each process?
- What is the waiting time for each process?
- What is the CPU utilization rate?

PRAVIN S. GAME

- Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process	Priority	Burst
P1	4	5
P2	1	3
P3	2	1
P4	2	7
P5	3	4

The processes are assumed to have arrived in the order P1, P2, P3, P4, P5, all at time 0.

- Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithms: FCFS, SJF, nonpreemptive priority (a larger priority number implies a higher priority), and RR (quantum = 2).
- What is the turnaround time of each process for each of the scheduling algorithms in part a?
- What is the waiting time of each process for each of these scheduling algorithms?
- Which of the algorithms results in the minimum average waiting time (over all processes)?

PRAVIN S. GAME

- The following processes are being scheduled using a preemptive, priority-based, round-robin scheduling algorithm.

Process	Priority	Burst	Arrival
P1	8	15	0
P2	3	20	0
P3	4	20	20
P4	4	20	25
P5	5	5	45
P6	5	15	55

Each process is assigned a numerical priority, with a higher number indicating a higher relative priority. The scheduler will execute the highest priority process. For processes with the same priority, a round-robin scheduler will be used with a time quantum of 10 units. If a process is preempted by a higher-priority process, the preempted process is placed at the end of the queue.

- Show the scheduling order of the processes using a Gantt chart.
- What is the turnaround time for each process?
- What is the waiting time for each process?

PRAVIN S. GAME

Think-Pair-Share

- What (if any) relation holds between the following pairs of algorithm sets?

- a. Priority and SJF
- b. Multilevel feedback queues and FCFS
- c. Priority and FCFS
- d. RR and SJF

- Suppose that a CPU scheduling algorithm favors those processes that have used the least processor time in the recent past. Why will this algorithm favor I/O-bound programs and yet not permanently starve CPU-bound programs?

PRAVIN S. GAME

- Which of the following scheduling algorithms could result in starvation?

- a. First-come, first-served
- b. Shortest job first
- c. Round robin
- d. Priority

PRAVIN S. GAME

REFERENCES

- Silberschatz, Galvin, Gagne, "Operating System Principles", 10th Edition
- Stallings W., "Operating Systems- internals and design principles", 9th Edition.
- Deitel, Deitel, Choffnes, "Operating System", 3rd edition, Pearson
- Andrew S. Tanenbaum; Modern Operating Systems; Prentice Hall of India Publication; 4th Edition